

ROPSCON: Linear BFT From Schnorr Threshold Signatures

Xuyang Liu , Zijian Zhang , Senior Member, IEEE, Haibin Zhang , Jiamou Liu ,
Meng Li , Senior Member, IEEE, Zhen Li , Member, IEEE, Mengmeng Zhou , Wei Shen ,
Peng Jiang , Cong Guo , and Liehuang Zhu , Senior Member, IEEE

Abstract—Existing linear Byzantine fault-tolerant (BFT) protocols heavily rely on Threshold Signatures (TS) to reduce communication complexity. The threshold signatures used and implemented in these BFT protocols should be non-interactive and all such schemes rely on heavy pairing operations. Threshold signatures using no pairings such as Schnorr TS are computationally more efficient but require multiple rounds of interaction and lack robustness. There has been a long-standing question on whether one could build more efficient BFT protocols from such pairing-free signatures. In this paper, we develop a novel approach that allows using threshold signatures like Schnorr TS in linear BFT protocols in an effective manner. In this approach, we can significantly reduce the interaction overhead caused by Schnorr TS while maintaining its computational efficiency. We instantiate a HotStuff-style BFT protocol called ROPSCON and show that ROPSCON achieves up to $2.2\times$ higher throughput compared to both HotStuff protocols using pairings as well as ones with directly aggregated signatures. The improvement is particularly evident in high-delay and low-bandwidth environments.

Index Terms—Distributed consensus, Byzantine fault tolerance, threshold signature.

I. INTRODUCTION

PROBLEM Overview and Motivation: Byzantine Fault-Tolerant (BFT) consensus mechanism has long been known as a crucial primitive for distributed environment with limited trust and adversarial threats. It is now even more prominent because it is widely regarded as a core building block for blockchain technologies [1], [2], [3]. This paper focuses on low communication complexity, linear BFT protocols under partially synchronous assumptions. Almost universally, these protocols rely heavily on Threshold Signatures (TS), which allow individual signatures shares (votes) from distinct replicas to be combined into a single constant-size signature (quorum proofs) for mutual verification. This aggregation technique dramatically reduces communication overhead compared to naive approaches that broadcast all individual signatures. A typical example is the HotStuff [4] protocol, which amalgamates features from SBFT [5] (collective signatures), Tendermint [6] (block locking), and from Casper [7] (pipelining techniques) to achieve a concise three-phase voting protocol.

Existing BFT systems leveraging TS can be divided into two categories: (1) using a collection of individual signatures in place of aggregated threshold signatures — a substitution that leads to significant bandwidth cost and signature validation overhead as the network expands. (2) using non-interactive but pairing schemes (e.g., threshold BLS [8], [9], [10]), which are computationally expensive due to the use of pairings.

This paper demonstrates pairing-free and computationally efficient Schnorr-based TS schemes can be used as alternatives to build linear BFT that in one way or another outperform those using the other existing patterns - although this is often considered challenging. Intuitively, existing Schnorr TS constructions [11], [12], [13], [14], [15] have several drawbacks.

The Robustness Problem. The most critical challenge of building BFT protocols using Schnorr TS is the lack of “true robustness.” In practical terms, this means that if even a single malicious replica submits an invalid signature share during the voting process, the entire signature generation fails. While prior schemes [12], [16] relied on strong synchronization to achieve robustness, these schemes cannot be used to build asynchronous BFT protocols.

Received 13 April 2026; revised 27 June 2026; accepted 11 August 2026. Date of publication 17 August 2026; date of current version 12 October 2026. This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 625B2022, Grant U2468205, Grant 62372149, Grant U23A20303, and Grant 62572168; in part by Anhui Provincial Natural Science Foundation under Grant 2508085MF151; and in part by China Scholarship Council under Grant 202506030203. Recommended for acceptance by Z. Xu. (Corresponding authors: Zijian Zhang; Peng Jiang.)

Xuyang Liu is with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China, and also with Shandong Key Laboratory of Energy Industry Internet Big Data Technology, Jinan, Shandong 250003, China (e-mail: liuxuyang@bit.edu.cn).

Zijian Zhang, Zhen Li, Peng Jiang, and Liehuang Zhu are with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China (e-mail: zhangzijian@bit.edu.cn; zhen.li@bit.edu.cn; pengjiang@bit.edu.cn; liehuangz@bit.edu.cn).

Haibin Zhang is with the Region Institute of Tsinghua University, Jiaxing, Zhejiang 314006, China (e-mail: bchainzhang@aliyun.com).

Jiamou Liu is with the School of Computer Science, The University of Auckland, Auckland 1010, New Zealand (e-mail: jiamou.liu@auckland.ac.nz).

Meng Li is with the School of Computer Science and Information Engineering, Hefei University of Technology, Hefei 230601, China (e-mail: mengli@hfut.edu.cn).

Mengmeng Zhou and Wei Shen are with the Beijing Academy of Blockchain and Edge Computing, Beijing 100081, China (e-mail: zhousmm@baec.org.cn; shenwei@baec.org.cn).

Cong Guo is with the National Meteorological Information Centre, Beijing 100081, China (e-mail: guoc@cma.gov.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TC.2026.3724280>, provided by the authors.

Digital Object Identifier 10.1109/TC.2026.3724280

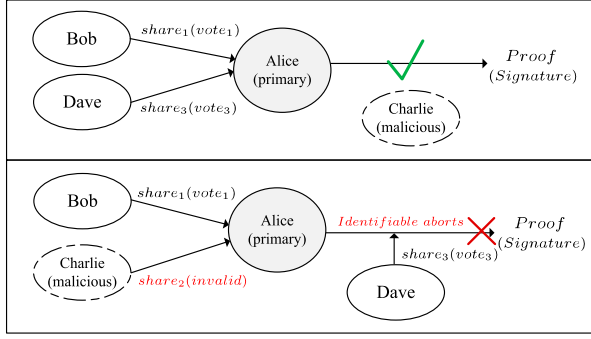


Fig. 1. An example for $n = 4$ (total number of replicas) and $t = 3$ (threshold of distinct shares required to construct a valid signature), the lack of true robustness in using Schnorr-based TS allows an adversary to disrupt the consensus proof construction by sending a malicious signature share.

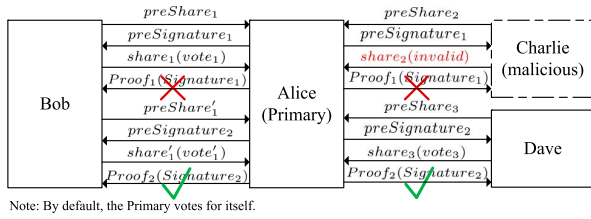


Fig. 2. Also with $n = 4$, $t = 3$, when proof construction through TS necessitates multiple interaction rounds, the signature process does not align with the single-round communication of proposal voting. Furthermore, to address the robustness challenge, we must consider signing in a session fails due to malicious replica interference.

Meanwhile, protocols like FROST [13] offer “identifiable aborts (IA)” rather than true robustness. While non-faulty replicas can identify which share caused the failure and abort the signing session, this creates a critical liveness vulnerability in BFT protocols: a single malicious replica can repeatedly disrupt consensus by submitting invalid signature shares (votes), effectively breaking the system’s liveness guarantees. We visualize the problem in Fig. 1.

The Multi-Round Interaction Challenge. Schnorr TS inherently require multiple communication rounds between participants. They typically involve a pre-signing phase where replicas generate and share presignature shares, followed by an actual signing phase where replicas compute signature shares based on the preprocessed values and the message to be signed. Some robust schemes such as ROAST [21] may require more interaction rounds due to the need for parallel sessions, or restarting the session when signing fails.

Such multi-round structure conflicts with the communication patterns expected by BFT protocols. Traditional BFT voting phases are designed around single-round interactions: the leader (primary) broadcasts a proposal, replicas respond with votes, and the leader aggregates these votes into a proof. The mismatch between Schnorr TS’s multi-round requirements and BFT’s single-round expectations creates significant integration challenges, as shown Fig. 2.

The Performance Paradox. Due to these challenges, simply calling existing Schnorr TS schemes in BFT protocols often results in worse performance than the alternatives they aim to

replace, which defeats the purpose of using computationally efficient signatures if overall protocol performance suffers.

Our Contributions. To our knowledge, we present the first linear BFT Consensus Protocol from Schnorr TS, called ROPSCON (Robust Partial Synchronous Schnorr-based Consensus), to exhibit the following characteristics:

- **Robustness using Schnorr TS:** We develop a novel algorithmic framework that enables the effective use of Schnorr TS in linear BFT’s proof construction. Leveraging ROAST’s signing model, ROPSCON create multiple sessions during proof construction. If a replica is identified as malicious during a session by submitting invalid signature shares, it is marked and excluded from all future sessions.
- **Redundancy Avoidance:** We explore semi-interactive Schnorr TS, delineating pivotal roles and properties, and extend them and into the consensus model. In ROPSCON, the signing process is not separated from consensus process. Each communication round serves dual purposes: advancing consensus and contributing to signature construction, which eliminates redundant communication steps. Additionally, we use minimal data structures to achieve multi-sessions management, ensuring proper initialization and avoiding conflicts.
- **Pipelining:** ROPSCON develops a new pipelining technique to augment the efficiency, as follows:
 - (1) The signing process and pre-signing process are streamlined across consecutive views. As one primary collects signature shares for current consensus round, the next primary initiates pre-signing phase for the subsequent round.
 - (2) The signing sessions are also pipelined. Each replica is responsible to not only participate in the current signing activities but also to proactively generate and provide presignature shares for future sessions.
- **Proactive View-Change:** ROPSCON enables the current primary to proactively initiate view changes when it becomes clear that sufficient valid votes cannot be collected in the current view. This action wakes up other replicas and resets the primary’s state, preventing the creation of unnecessary sessions and saving idle time.

Table I shows a comparison of linear BFT protocols utilizing distinct signature schemes with ROPSCON based on the criteria previously established. Both proof verification time and proof size were obtained through experimental measurements. Instead of using the original data, we simplified it into a scaled form to make comparisons easier.

We present ROPSCON in HotStuff notations, since most state-of-art linear BFT protocols employing TS to construct

¹Best case refers to when no malicious signature share interferes successfully during the proof construction, typically when all replicas are non-faulty, or all faults are crash faults, or after malicious replicas (sending invalid shares to interfere) have been identified and excluded through our robustness framework. We prove the best-case scenario eventually occurs within finite number of views if faults do not exceed tolerance.

TABLE I
COMPARISON OF LINEAR BFT PROTOCOLS (PARTIALLY SYNCHRONOUS) UTILIZING DISTINCT SIGNATURE SCHEMES WITH ROPSCON

	Proof verification time (20 replicas)	Proof size (20 replicas)	IA	Safety & Liveness	Robustness	Interaction rounds with the current primary per view (best case)
Linear BFT protocols [4], [17], [18], [19] + ed25519 (using a collection of individual signatures)			-	✓	✓	Send once, receive once
Linear BFT protocols + BLS TS [8], [9]			-	✓	✓ or ✗	Send once, receive once
Linear BFT protocols + IA Schnorr TS [13], [14], [20] (direct call, no adaptation to the consensus process)			✓	✓	✗	Send twice, receive twice
Linear BFT protocols + Robust Schnorr TS (direct call) [12], [21], [15]	- (incompatible)	-	-	-	-	-
ROPSCON (this paper)			✓	✓	✓	Send once, receive once

proofs are derived from HotStuff variants, making it easy to extend ROPSCON to similar paradigms. Our experimental results show ROPSCON achieves higher (up to 2.2x) throughput compared to both HotStuff protocols using pairings as well as ones using collections of individual signatures. The improvement is particularly evident in high-delay and low-bandwidth environments. The protocol maintains robustness against invalid signature share interference during proof construction with minimal performance impact. By avoiding redundancy and using pipelining, ROPSCON significantly increases throughput and performs well under regular conditions.

II. RELATED WORK

BFT. In 1982, Lamport et al. introduced the Byzantine Generals Problem [22], framing the challenge of achieving consensus despite Byzantine failures, but initial solutions in synchronous settings were communication-intensive. Soon after, Fischer et al. showed that in asynchronous networks [23], reaching consensus deterministically is impossible with even a single fault [24], while Ben-Or established fault tolerance bound [25]. Dwork et al. proposed the partially synchronous model [26], offering a new approach, leading to Castro and Liskov's practical solution in 1999 [27], laying groundwork for many future BFT systems.

More recent developments in BFT protocols have focused on enhancing performance, with one potential avenue for improvement being the reduction of complexity. SBFT [5] and HotStuff [4] have both advanced in this regard using TS, with HotStuff also introducing a basic linear structure framework and bringing responsiveness for simplifying BFT protocols, thus facilitating the generation of numerous variants [17], [18], [19], [28], [29], [30]. Additionally, research involving Directed Acyclic Graphs [31], [32], [33] has furthered progress in mitigating the throughput challenge in large-scale BFT systems. Some of their building blocks also require TS for implementation. Existing BFT systems use collections of individual signatures as an alternative when deploying TS, or rely on non-interactive but pairing schemes such as BLS [8], [9], [10]. In this paper, we focus on optimizing the performance of linear BFT using more computationally efficient TS.

Threshold Signatures (TS). Threshold cryptography enhances the security, reliability and decentralization of cryptographic key management by protecting against key theft and accidental loss [34], [35], [36]. The main challenge is to construct compact TS schemes where the signature size does not depend on n

and t . An ideal scheme should have constant-size public key and signature that matches the underlying single-signer scheme, which can bring benefits such as TS scheme can be a drop-in replacement in systems already supporting the underlying scheme or verifiers do not need to know the specific implementation details of TS scheme, etc.

While ECDSA [37], [38], [39] is widely supported, compact threshold ECDSA is intricate due to the inherent complexities of their algebraic structure. Multi-round protocols using advanced MPC [40] techniques are needed. In contrast, threshold Schnorr and BLS schemes are simpler. Despite having fewer interaction rounds, BLS's reliance on pairing operations makes it computationally intensive and less efficient, leading to the preference for Schnorr in efficiency-seeking protocols. FROST [13], a state-of-the-art Schnorr TS scheme, is distinguished by its semi-interactive nature (i.e., performing interactive preprocessing round before knowing the message), optimal round efficiency, and support for arbitrary selections of t and n . Several FROST variant schemes [14], [21], [41] have been proposed subsequently, which offer modifications in presignature share aggregation and duplicate checking.

Robustness in Schnorr TS. The pursuit of robust Schnorr TS, where valid signatures are produced even in the presence of disruptive signers, has been challenging. The initial designs utilized distributed key generation (DKG) protocols [16] during each signing session to create the random group element in Schnorr signatures. This ensured continuity if some signers disappeared mid-session. Prior schemes like TSch [16], new-TSch [12] and ATSSIA [42] used this approach and provided robust constant-round signing. However, disadvantages like conflating benign failures, the reliance on strong synchronous network and endpoint assumptions hindered adoption. While SPRINT [15] removes the reliance on strong synchronization, it still requires atomic broadcast channels that all participants must access during signing.

Recent developments like FROST [13] and Lindell's 3-round protocol [43] shifted focus from DKG-based designs to simpler, more efficient protocols. These newer designs, while not inherently robust, guarantee IA: if a signing session fails due to a malicious signer, honest signers can identify and exclude it and restart the process. On this basis, ROAST [21] framework has been proposed. It can provide robustness for any given semi-interactive TS scheme with IA and follows asynchronous assumptions, providing practical value. This makes it possible to explore BFT from Schnorr TS.

III. PRELIMINARIES

A. Semi-Interactive TS

Overview. TS are a type of t -of- n signatures. After the initial key generation, which involves a group of n signers, denoted as $\mathcal{S}_1, \dots, \mathcal{S}_n$, any subgroup $\{\mathcal{S}_i\}_{i \in T}$ of t signers can interactively generate a signature. This subgroup is identified by an index set $T \subseteq \{1, \dots, n\}$ with $|T| = t$. Here, n and t are fixed parameters for key generation. The signature is valid under a threshold public key, representing the entire group.

We classify a TS scheme as semi-interactive if signing consists of two discrete steps: preprocessing and actual signing. In the preprocessing round, each signer independently performs preprocessing without knowledge of the forthcoming message or the set of participating signers. Every signer then broadcasts the public output of each step to all others. We assume the presence of an implicitly provided security parameter λ , and the bitstring encoding of an indexed set includes an encoding of the index set T . A semi-interactive TS scheme $\Sigma = (\text{Gen}, \text{PreRound}, \text{PreAgg}, \text{SignRound}, \text{SignAgg}, \text{Verify})$ includes probabilistic polynomial-time algorithms as follows:

- $(PK, (sk_1, \dots, sk_n)) \leftarrow \langle \text{Gen} \rangle$: The key generation algorithm $\text{Gen} = (\text{Gen}_1, \dots, \text{Gen}_n)$ is a collection of interactive algorithms run by signers. Each signer $\mathcal{S}_i$ runs $\text{Gen}_i(n, t)$, which returns its secret key sk_i and a public-key object PK common to all signers.
- $(state_i, \rho_i) \leftarrow \text{PreRound}(PK)$: The preprocessing algorithm is run by signer $\mathcal{S}_i$. It takes PK as input and outputs a secret state $state_i$ and a presignature share ρ_i .
- $\rho \leftarrow \text{PreAgg}(PK, \{\rho_i\}_{i \in T})$: The deterministic presignature aggregation algorithm takes PK and a set $\{\rho_i\}_{i \in T}$ of presignature shares as input. It outputs a presignature ρ .
- $\sigma_i \leftarrow \text{SignRound}(sk_i, PK, T, state_i, \rho, m)$: The signature share algorithm is run by signer $\mathcal{S}_i$. It takes sk_i , PK , an index set T of signers, a secret $state_i$, a presignature ρ , and a message m as input. It outputs a signature share σ_i .
- $\sigma \leftarrow \text{SignAgg}(PK, \rho, \{\sigma_i\}_{i \in T}, m)$: The deterministic signature aggregation algorithm takes PK , ρ , a signature shares set $\{\sigma_i\}_{i \in T}$, and m as input. It outputs a signature σ .
- $b \leftarrow \text{Verify}(PK, m, \sigma)$: The verification algorithm takes PK , m , and σ as input. It outputs a boolean b , where $b = \text{true}$ means that the signature is valid, and vice versa.

A coordinator $\mathcal{C}$ is required who is responsible for managing the signing sessions of Σ , besides broadcasting messages, and can operate on the same machine as one of the signers. $\mathcal{C}$ is considered semi-trusted, meaning it is trusted only for robustness, but not for unforgeability, allowing for an optimistic approach in choosing $\mathcal{C}$ in practice: if a selected $\mathcal{C}$ proves unreliable, it can be replaced. The role and characteristics of the coordinator allow it to be associated with the leader in consensus [44]. Therefore, in designing ROPSCON, we assigned the functions of coordinator and leader to a specific signer/replica, referred to as “cleader” in this paper.

Share Validation. An adversary aims to prevent signers from obtaining a valid signature on m given as input to the coordinator. Given a set $F \subseteq [n]$ of $|F| = f$ indices of malicious signers. We assume the adversary controls f signers both during key generation and signing. Such behaviour is consistent with general assumptions and can be directly integrated to the consensus threat model. To identify malicious shares in a signing session, an extra deterministic algorithm is required.

- $b \leftarrow \text{ShareVal}(PK, T, i, \rho, \rho_i, \sigma_i, m)$: The share validation algorithm takes PK , the index i of $\mathcal{S}_i$, the presignature ρ , the presignature share ρ_i and the signature share σ_i of signer $\mathcal{S}_i$ as input. It returns a boolean b which is true if and only if ρ_i and σ_i are valid contributions of $\mathcal{S}_i$.

Security. The main security properties of TS required for our techniques can be succinctly summarised as follows:

Identifiable Aborts (IA): In the IA-CMA game [13], the adversary controls all but one signer and can initiate concurrent signing sessions. The adversary wins if: (1) In some session, malicious signers submit shares that pass ShareVal but lead to an invalid signature. (2) The honest signer outputs shares that fail ShareVal . A scheme is IA-CMA secure if the adversary’s advantage in winning this game is negligible.

Unforgeability: A TS scheme is unforgeable if no adversary controlling less than t signers can produce a valid signature on a message never properly signed.

As we default to use FROST3 [21] as the underlying TS scheme, which has been proven to fulfil identifiable aborts and unforgeability properties, we omit further details here.

Additional Properties. Besides inheriting the security properties of the underlying TS scheme, ROAST provides two properties essential for BFT: asynchrony (signature progress depends only on message arrival rather than timing assumptions), and robustness (with up to f malicious signers, the coordinator eventually outputs a valid signature on input m). These properties are important because traditional robust TS schemes assume synchrony and are vulnerable to timing manipulation under partial synchrony or asynchrony. Moreover, unforgeability and IA alone do not prevent malicious signers from disrupting proof construction by submitting invalid shares within a view.

B. FROST, FROST3 and ROAST

FROST [13] is a semi-interactive Schnorr TS scheme that optimizes round efficiency using a preprocessing round before the signing. It supports arbitrary choices of threshold parameters t and n , including configurations with $t - 1 \geq n/2$ that ensure unforgeability even when f malicious signers form a dishonest majority ($n/2 \leq f \leq t - 1$). However, FROST lacks robustness as its signing protocol fails if a malicious signer participates. This results from the design trade-off of eliminating DKG during signing to achieve a simpler and more efficient protocol compared to prior schemes.

FROST3 [21] is a modified version of FROST that enables aggregation of presignature shares and IA. It also eliminates the duplicate check present in some FROST variants [14] that may prevent IA by requiring honest signers to abort if two signers submit the same presignature share $\rho_i = \rho_j$ for $i \neq j$.

We provide the full algorithm of FROST3 in Appendix A of the supplementary material.

ROAST [21] is a framework that transforms any given semi-interactive TS scheme with IA (such as FROST3) into a robust and asynchronous TS scheme. To sign a message m , the coordinator $\mathcal{C}$ maintains a set R of “responsive” signers that have correctly responded to all previous signing requests. Whenever $|R| \geq t$, $\mathcal{C}$ initiates a new signing session with the signers in R by requesting them to provide shares. $\mathcal{C}$ aggregates the shares if they are valid, or otherwise marks unresponsive signers as malicious. This continues until $\mathcal{C}$ obtains t valid shares in some session and can output a valid TS. As ROAST initiates multiple concurrent signing sessions of the underlying TS scheme Σ , ROAST is unforgeable if Σ is unforgeable under concurrent signing sessions.

IV. PROBLEM STATEMENT

We first embed the pivotal roles, assumptions and properties from the previous section to consensus.

A. System Model

The system is presumed to comprise n replicas that process transactions from clients to ensure global consistency of operation logs. A semi-interactive IA-CMA-secure TS scheme $\Sigma = (\text{Gen}, \text{PreRound}, \text{PreAgg}, \text{SignRound}, \text{ShareVal}, \text{SignAgg}, \text{Verify})$ with unforgeability is required (in ROPSCON, we default to using FROST3 as the Σ). Each replica r possesses a public/private key pair (PK, sk_r) , generated via Gen (different methods can be used to create this setup. E.g., a suitable distributed DKG protocol [14] or simply a trusted dealer). All authorized replicas are interconnected to form a consensus group. Broadcasting a message implies sending the same message to some replicas in a point-to-point manner. Within a period referred to as a *view*, a replica serves as the **cleader** to drive the protocol for one or more views. We assume a partially synchronous network [26]. After some unknown global stabilization time (GST), messages between two non-faulty replicas can be transmitted within a known bound Δ .

B. Threat Model

Assume a set $F \subseteq [n]$ of $|F| = f$ indices of faults that are controlled by an adversary or down, where $f < n/3$. For simplicity, we set $n = 3f + 1$, i.e., at least $n - f = 2f + 1$ replicas are non-faulty. The malicious replicas exhibit byzantine faults [22] with arbitrary behavior: they can collude, produce arbitrary values, eavesdrop, selectively delay or omit messages, and send conflicting information, but cannot compromise the cryptographic primitives, prevent point-to-point communications between non-faulty replicas, or indefinitely delay the occurrence of GST. Non-faulty replicas strictly follow the protocol specification. A BFT protocol needs to satisfy the following properties:

- **Safety.** All non-faulty replicas have the the same ordered log of transactions.

- **Liveness.** A correct transaction is eventually delivered and committed by all non-faulty replicas.

We also aim to achieve **Robustness** - for partially synchronous BFT that utilize TS, we give the following definition:

- **(n, t, f') -Robustness.** For any view with t non-faulty replicas and up to f' malicious replicas, after GST, there exists a bounded time period such that a non-faulty cleader can construct a valid consensus proof for a valid proposal ($t = 2f + 1$ in ROPSCON).

A consensus protocol is considered robust if, for all $f' \leq n - t$, it is (n, t, f') -robust. We then can distinguish three cases:

Normal operation. If malicious replicas $\leq f$, robustness is crucial for ensuring consensus among non-faulty ones, maintaining safety and liveness.

Partial failure. If $f < \text{malicious replicas} \leq n - f$, they can prevent consensus among non-faulty replicas, breaking safety and liveness. The unforgeability of TS scheme prevents direct consensus proof constructions among malicious replicas (fooling non-faulty replicas into conflicting consensus is possible).

Game over. If malicious replicas $> n - f$, unforgeability is lost, allowing arbitrary malicious consensus.

C. Design Goals

Fig. 3 provides a Σ -based robust algorithmic framework for consensus proof construction, applicable to most leader-rotation linear BFT protocols, with ROAST as its multi-session signing model (Section III-B, paragraph 3). To implement a BFT protocol on top of it without redundancies, we need to:

- (1) Ensure voting and proof construction processes per consensus phase be completed solely within the multi-session signing and interaction outlined in the green dashed box. Each communication round should serve dual purposes: advancing consensus and contributing to signature construction, eliminating redundant communication steps.
- (2) Incorporate a pipelined methodology to maximize parallelization degree in processes outlined by the purple dotted box, so that the system should not block on pre-signature generation once entering a new view, and replicas should not block on preround when entering a new session.
- (3) Create and manage session and TS-related variables correctly, including proper initialization and conflict avoidance.

In the next section, we present our protocol using HotStuff notations since most state-of-art linear BFT protocols employing TS to construct proofs are derived from HotStuff variants, making it easy to extend ROPSCON to similar paradigms. We show how to build (or replace the existing signature framework to achieve) a robust leader-rotation linear BFT protocol using this framework and achieve the three objectives.

V. ROPSCON

Built upon an IA-CMA-secure TS scheme Σ with unforgeability and using Fig. 3 as its algorithmic framework, ROPSCON addresses the State Machine Replication (SMR)

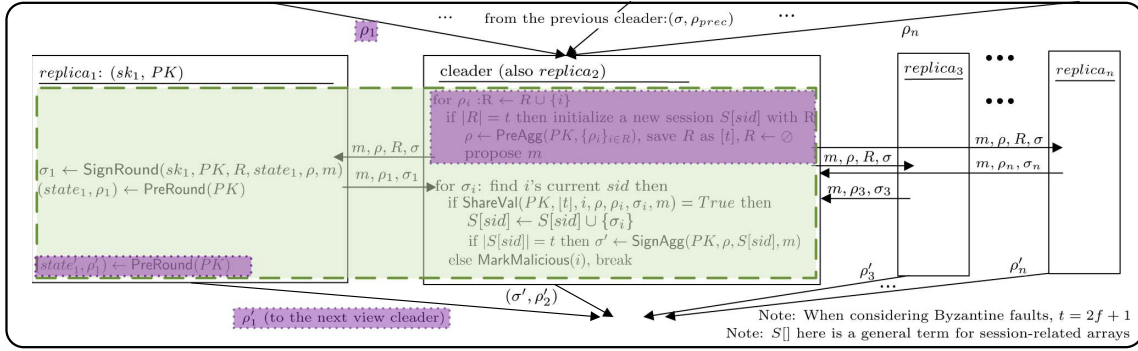


Fig. 3. The algorithmic framework.

problem [45] which aims to reach consensus on a growing log of transactions from clients. It functions in a series of views, each with a monotonically increasing number and has a unique dedicated cleader known to all. Replicas stores a tree of pending transactions as its local data structure, with each node containing a (or a batch of) proposed transaction, associated metadata, and a parent link. The branch led by a node represents the path from the node to the tree root via parent links. During execution, a node becomes committed when the cleader collect votes from $n - f = 2f + 1$ replicas to form a valid TS (proof). This node, along with the signature, is confirmed by the two subsequent views before it can be committed, forming a monotonically growing branch.

ROPSCON leverages the robust multi-session framework, enabling the effective use of Σ in proof construction. It maintains robustness in the presence of invalid votes (shares), though it comes at the cost of a slight increase in communication overhead; however, it is a trade-off that yields the computational efficiency advantages of pairing-free signatures. Moreover, ROPSCON employs a two-level pipelining strategy to achieve a high degree of parallelism, specifically:

Pipelining of the Signature Process: The signing phase and pre-signing phase are streamlined across consecutive views. As one cleader collects signature shares for the current view's signature (proof), the cleader of the subsequent view initiates the pre-aggregation of presignature shares, ensuring a non-stop flow of transaction processing and signature formation across views and reducing idle times.

Pipelining of Signing Sessions: In parallel with pipelining signature process, ROPSCON also pipelines the signing sessions. Each replica is responsible to not only participate in the current signing activities but also to proactively generate and provide presignature shares for future sessions. This dual responsibility guarantees the protocol is always prepared for next signing session without delay.

Moreover, ROPSCON achieves redundancy avoidance and correct variable management; the particulars of which will be delineated in protocol description.

A. Data Structures and Basic Utilities

Messages. A message m has a type ($m.type$) determined by the current protocol type, a proposed node ($m.node$) and a

view number ($m.view$). The $m.type$ is used to identify whether the protocol is currently in a state protocol or a view-change protocol, categorised as: STATE and VIEWCHANGE.

Communications. A communication has fields: $commun.m$ to carry a message m , $commun.sig$ to carry a signature. A $commun$ from a replica includes a presignature share ($commun.pshare$) and a signature share ($commun.share$) signed over m . A $commun$ sent by cleader includes a set of replicas response to m ($commun.resp$) for signature verification and a presignature pre .

Signatures. A signature sig has a (full) signature $sig.\sigma$ used by the cleader to carry a (full) signature and by replicas to carry the signature from the highest view. $sig.m$ carries the signed messages (note that the meanings of $sig.m$ and $commun.m$ are different).

Tree, branches and nodes. A tree is a hierarchical data structure where each node has one parent, except for the root, which has no parent. A branch is a sequence of connected nodes that starts from the root node and ends at a leaf node in the tree. Two branches are conflicting if neither one is an extension of the other. A node $node$ contains its parent $node.parent$, the transaction(s) $node.command$ and the signature proof of the previous node $node.proof$. Two nodes are conflicting if the branches led by them are conflicting.

The basic utilities and local variables for replica i are specified in Algorithm 1. CreateNode, CreateMsg, CreateSig, CreateCommun are used to create the underlying data structures. MarkMalicious is used to mark malicious replicas and add them to $Malic$, which helps to improve consensus efficiency and reduce parallel sessions.

Note that $Malic$ is maintained locally and is only used to exclude suspected replicas from the local replica's future signing sessions. A replica is added to $Malic$ only when it sends an unsolicited response that violates the current session state (Algorithm 2, Line 5), or when its signature share fails ShareVal (Algorithm 2, Line 8). Message delay alone does not cause a non-faulty replica to be incorrectly marked by another non-faulty replica. A non-faulty replica does not send unsolicited responses: its delayed message is either delivered within its designated session and processed, or arrives too late and is no longer processed. If a message reaches the ShareVal step, it has already passed isMatch and corresponds to an existing

Algorithm 1 Local variables and basic utilities for replica i **Local variables:**

$\triangleright$ The following variables are initialised only at the start of the consensus
 $state_c \leftarrow \perp$ $\triangleright$ Record the secret state of the current view, must be private.
 $state_n \leftarrow \perp$ $\triangleright$ Record the secret state of the next view, must be private.
 $view \leftarrow 1$ $\triangleright$ Incremented either by reaching consensus or by completing a view change.
 $curProposal \leftarrow \perp$ $\triangleright$ Record the current proposal (cleader).
 $topSig \leftarrow \perp$ $\triangleright$ Record the highest known valid sig.
 $commitSig \leftarrow \perp$ $\triangleright$ Record the highest known valid sig, whose corresponding node is about to be committed.
 $\triangleright$ The following variables are initialised after entering the view-change protocol
 $Malic \leftarrow \emptyset$ $\triangleright$ Replica i is known to be malicious $i \in M$.
 $\triangleright$ The following variables are initialised at the start of each consensus view
 $topsid \leftarrow 0$ $\triangleright$ Record the newest session id during the threshold signing process
 $Pshare[] \leftarrow \text{array}(n)$ $\triangleright Pshare[i]$ is the latest presignature share of S_i .
 $Sid[] \leftarrow \text{array}(n)$ $\triangleright Sid[i]$ is the session that includes replica i .
 $Share[] \leftarrow \text{array}(f+1)$ $\triangleright Share[sid]$ is the set of signature shares for session sid
 $Ra[] \leftarrow \text{array}(f+1)$ $\triangleright Ra[sid]$ records the replicas that responded in session sid
 $Pre[] \leftarrow \text{array}(f+1)$ $\triangleright Pre[sid]$ is the presignature of session sid
 $\triangleright$ The following variable is initialised at the start of each signing session
 $R \leftarrow \emptyset$ $\triangleright$ Replica i is responsive if $i \in R$

<pre> 1: function CreateNode (parentNode, command, proof) 2: node.parent $\leftarrow$ parentNode, node.cmd $\leftarrow$ command, node.proof $\leftarrow$ proof 3: return node 4: function CreateMsg (type, node, view) 5: m.type $\leftarrow$ type, m.node $\leftarrow$ node, m.view $\leftarrow$ view 6: return m 7: function MarkMalicious (m, sig, share) 8: Malic $\leftarrow$ Malic $\cup \{i\}$ 9: if Malic > f then Enter the view-change protocol 10: function isMatch (m, type, v) 11: return (m.type = type) $\wedge$ (m.view = v) </pre>	<pre> 12: function CreateCommun (m, sig, share, pshare, resp, pre) 13: commun.m $\leftarrow$ m, commun.sig $\leftarrow$ sig 14: commun.share $\leftarrow$ share, commun.pshare $\leftarrow$ pshare 15: commun.resp $\leftarrow$ resp, commun.pre $\leftarrow$ pre 16: return commun 17: function CreateSig (σ, m) 18: sig.σ $\leftarrow$ σ, sig.m $\leftarrow$ m 19: return sig 20: function isValid (node) 21: return ((node is a descendant of commitSig.m.node) $\vee$ ((node.proof.m.view > commitSig.m.view) $\wedge$ (Verify(PK, node.proof.m, node.proof.σ))) </pre>
--	--

designated signing session. Since non-faulty replicas follow the protocol and submit valid shares, and malicious replicas cannot compromise the cryptographic primitives under the IA-CMA security of the underlying scheme, the probability that a share from a non-faulty replica fails ShareVal is negligible.

Nevertheless, retransmitted or replayed messages may be misclassified as unsolicited if the same vote is received again after the replica has already been added to the response set. This issue is not caused by the signature validation mechanism and can be addressed by standard anti-replay techniques, such as binding each vote to a session identifier and/or including a nonce, so that duplicate or stale messages are ignored rather than treated as malicious behavior. For simplicity, we treat such anti-replay protection as a standard implementation-level mechanism and omit it from the pseudocode. Accordingly, the subsequent discussion assumes that replayed messages are detected and discarded.

A faulty cleader may arbitrarily modify its own local $Malic$ and exclude non-faulty replicas from its own future sessions. This behavior can delay progress in the views led by this faulty cleader, but it does not give the adversary additional power beyond a standard Byzantine leader that simply ignores received messages. Since $Malic$ is maintained locally, such arbitrary marking cannot affect the $Malic$ sets of non-faulty replicas or future non-faulty cleaders.

When number of marked replicas exceeds the system's tolerance limit, the cleader attempts to enter the view-change protocol (this behaviour is reasonable given that the actual number of malicious replicas must be less than or equal to marked replicas. In addition, it allows the cleader to enter the view-change protocol early without waiting for a timeout when it's clear enough votes cannot be collected to reach consensus).

The protocol also requires two checking functions, where isMatch is used to isolate messages in different views and of different types, so as to avoid (1) creating a signature with outdated shares (2) treating view-change messages as if they were messages of the normal state protocol, etc., which would cause confusion in the consensus protocol. isValid is used to check whether the proposed node satisfies the safety or liveness rules. It returns true if the node extends from the $commitSig.m.node$ (safety), or if the $view$ in $node.proof.m$ is greater than the $view$ in $commitSig.m$ (liveness).

B. Protocol Specification

Normal State Protocol. The protocol outlined in Algorithm 2 operates as a continuous loop, segmented by views. Within each view, a designated replica serves as the cleader assuming the role of the consensus leader and the coordinator for the TS, whose tasks include (1) packaging the transaction(s) with the highest known $commitSig$ into a proposal and broadcast it; (2) maintaining a set R of responsive replicas that have responded to all previous signing requests by sending valid presignature

Algorithm 2 Normal State Protocol

```

1: Initialization:  $view, topSig, commitSig$ 
2: for  $view \leftarrow 1, 2, 3 \dots$  do
3:   as Cleader( $view$ ) do  $\triangleright$  Signing phase
4:     upon receiving  $commun$  from  $replica_i, i \notin Malic$  and  $isMatch(commun.m, STATE, view)$ :  $\triangleright$  Receiving votes from replicas
5:       if  $i \in R$  then  $MarkMalicious(i)$ , break  $\triangleright$  Avoidance of unsolicited
6:       if  $Sid[i] \neq \perp$  then  $sid \leftarrow Sid[i]$   $\triangleright$  If not an initial pre-signing message, find the corresponding signing session
7:        $T \leftarrow Ra[sid], \rho \leftarrow Pre[sid], \rho_i \leftarrow Pshare[i]$   $\triangleright$  Fetch the data required to verify the signature share
8:       if  $\neg ShareVal(PK, T, i, \rho, \rho_i, commun.share, curProposal)$  then  $MarkMalicious(i)$ , break  $\triangleright$  Invalid share
9:        $Share[sid] \leftarrow Share[sid] \cup \{commun.share\}$   $\triangleright$  Store the valid signature share
10:      if  $|Share[sid]| = 2f + 1$  then  $\sigma \leftarrow SignAgg(PK, \rho, Share[sid], curProposal)$   $\triangleright$  Enough to construct a new proof
11:       $topSig \leftarrow createSig(\sigma, curProposal)$   $\triangleright$  Package the signature into a proof
12:       $send CreateCommun(\perp, topSig, \perp, \perp, \perp, \perp)$  to Cleader( $view + 1$ )  $\triangleright$  Cleader's work in his own view is completed
13:       $Pshare[i] \leftarrow commun.pshare, R \leftarrow R \cup \{i\}$   $\triangleright$  Store the newest presignature share from replica_i
14:      if  $|R| = 2f + 1$  then  $topsid \leftarrow topsid + 1$   $\triangleright$  Initiate a new session if there are  $2f + 1$  responsive replicas
15:       $\rho \leftarrow PreAgg(PK, \{Pshare[i]\}_{i \in R})$ 
16:      for  $i \in R$  do  $Sid[i] \leftarrow topsid$   $\triangleright$  Mark the replica as responsive
17:       $broadcast CreateCommun(curProposal, \perp, \perp, \perp, R, \rho)$   $\triangleright$  Send the presignature to replicas
18:       $Ra[topsid] \leftarrow R, Pre[topsid] \leftarrow \rho$   $\triangleright$  Remember the replicas and the presignature
19:      Initialization:  $R$ 
20:   as a replica ( $replica_i$ ) do
21:      $\triangleright$  Voting phase, which will not be triggered if the cleader constructs a valid proof in the first session
22:     upon receiving  $commun$  from Cleader( $view$ ),  $isMatch(commun.m, STATE, view)$ ,  $commun.m = curProposal$  and  $i \in commun.resp$ :
23:        $\sigma_i \leftarrow SignRound(sk_i, PK, commun.resp, state_c, commun.pre, curProposal)$ 
24:        $(\rho_i, state_c) \leftarrow PreRound(PK)$   $\triangleright$  Update the secret state of current view
25:        $send CreateCommun(curProposal, \perp, \sigma_i, \rho_i, \perp, \perp)$  to Cleader( $view$ )  $\triangleright$  Send the signature share if responsive
26:      $\triangleright$  When a proposal is received from the cleader of a higher view
27:     upon receiving  $commun$  from Cleader( $view + 1$ ),  $commun.m \neq \perp$  and  $isMatch(commun.m, STATE, view + 1)$ :
28:        $n_0 \leftarrow commun.m.node, n_1 \leftarrow n_0.proof.m.node, n_2 \leftarrow n_1.proof.m.node, n_3 \leftarrow n_2.proof.m.node$ 
29:       if ( $isValid(n_0)$ ) then  $curProposal \leftarrow commun.m$   $\triangleright$  If the proposal is valid
30:        $reset$  timer and  $enter$  view  $v + 1$   $\triangleright$  Simultaneously entering the voting phase
31:        $state_c \leftarrow state_n$   $\triangleright$  Look up the initial presignature share prepared for the current view
32:        $(\rho'_i, state_n) \leftarrow PreRound(PK)$   $\triangleright$  Initial presignature share for then next view
33:       if  $i \in commun.resp$  then  $\triangleright$  Replicas not included in the commun.resp set are exempt from sending the share
34:          $\sigma_i \leftarrow SignRound(sk_i, PK, commun.resp, state_c, commun.pre, curProposal)$ 
35:          $(\rho_i, state_c) \leftarrow PreRound(PK)$   $\triangleright$  Update the secret state of current view
36:          $send CreateCommun(curProposal, \perp, \sigma_i, \rho_i, \perp, \perp)$  to Cleader( $view$ )
37:       else  $send CreateCommun(curProposal, \perp, \perp, \rho_i, \perp, \perp)$  to Cleader( $view$ )  $\triangleright$  Presignature share only
38:        $send CreateCommun(\perp, \perp, \perp, \rho'_i, \perp, \perp)$  to Cleader( $view + 1$ )  $\triangleright$  Send the initial presignature share to the next cleader
39:       if ( $n_0.parent = n_1$ )  $\wedge$  ( $Verify(PK, n_0.proof.m, n_0.proof.\sigma)$ ) then  $topSig \leftarrow n_0.proof$   $\triangleright$  Two confirmations required
40:       if ( $n_1.parent = n_2$ )  $\wedge$  ( $Verify(PK, n_1.proof.m, n_1.proof.\sigma)$ ) then  $commitSig \leftarrow n_1.proof$ 
41:       if ( $n_2.parent = n_3$ )  $\wedge$  ( $Verify(PK, n_2.proof.m, n_2.proof.\sigma)$ ) then  $Execute$   $n_3.command$ ,  $respond$  to clients
42:   as Cleader( $view + 1$ ) do  $\triangleright$  Initial Pre-signing phase
43:     Initialization:  $topsid, Pshare, Sid, Share, Ra, Pre, R$ 
44:     upon receiving  $commun$  from  $replica_i, i \notin Malic$  and  $\neg isMatch(commun.m, VIEWCHANGE, view + 1)$ :
45:        $Pshare[i] \leftarrow commun.pshare, R \leftarrow R \cup \{i\}$   $\triangleright$  Store the initial presignature share from replica_i
46:       if  $|R| = 2f + 1$  then  $topsid \leftarrow topsid + 1$   $\triangleright$  Initiate the first session if there are  $2f + 1$  responsive replicas
47:        $\rho \leftarrow PreAgg(PK, \{Pshare[i]\}_{i \in R})$ 
48:       for  $i \in R$  do  $Sid[i] \leftarrow topsid$   $\triangleright$  Mark the replica as responsive
49:        $wait$  for  $commun$  from Cleader( $view$ ),  $isMatch(commun.sig.m, STATE$  or  $VIEWCHANGE, view)$ 
50:       if ( $Verify(PK, commun.sig.m, commun.sig.\sigma) \wedge (commun.sig.m.view > topSig.m.view)$ ) then
51:          $topSig \leftarrow commun.sig$ 
52:          $leaf \leftarrow CreateNode(topSig.m.node, command$  or  $commands$  from clients,  $topSig)$   $\triangleright$  Create a leaf node
53:          $curProposal \leftarrow CreateMsg(STATE, leaf, view + 1)$   $\triangleright$  Package the node
54:          $broadcast CreateCommun(curProposal, topSig, \perp, \perp, R, \rho)$   $\triangleright$  Propose
55:          $Ra[topsid] \leftarrow R, Pre[topsid] \leftarrow \rho$   $\triangleright$  Remember the replicas and the presignature
56:         Initialization:  $R$ 

```

shares. Once R contains $2f + 1$ signers, the cleader initiates a new signing session i.e., ask each $replica_i$ for $i \in R$ to respond with a valid signature share σ_i . The cleader record valid shares, and marks senders of invalid shares as malicious. This continues until the cleader obtains $2f + 1$ valid shares in some session

to produce a valid signature to construct a valid proof of the current proposal (node). The proof will be sent to the next cleader to wake it up and proceed to the next view (Line 47). We have proved the upper bound of concurrent sessions is $f + 1$, the same as ROAST.

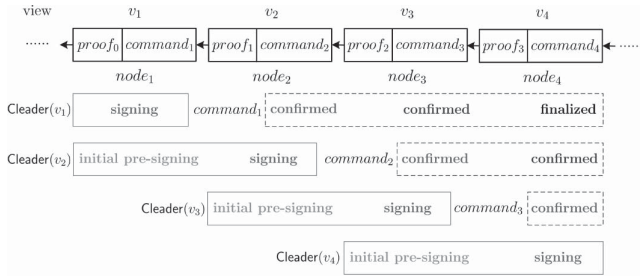


Fig. 4. Pipelining of the signature process.

After receiving a valid proposal, the replica sends its vote to the cleader (and also send the initial presignature share to the next cleader, which is related to our pipelining strategy and will be explained later). Note that replicas not included in the *commun.resp* set are exempt from executing the signature share algorithm based on *commun.p*, effectively remaining in the initial pre-signing phase. Then the replica enters the voting phase and resets the timer. Simultaneously, it confirms the nodes that have already provided valid proofs and finalizes the transaction that meets the commitment conditions. During the voting phase, the replica continuously waits for the cleader's presignature and returns the corresponding signature share. This process continues until a valid proposal is received from the cleader of a higher view (Line 25) or a timer timeout initiates the view-change protocol.

The succession of cleaders is determined by a rotation policy that caters to specific requirements, e.g. a rolling mode to maintain the chain quality. It is essential to ensure that consecutive cleaders in *view* and *view + 1* are not instantiated by the same replica to allow for the concurrent execution of the TS phases. This concurrent execution is critical for the pipelining of the signature process, as illustrated in Fig. 4.

For instance, during view v_1 , $\text{Cleader}(v_1)$ is responsible for collecting signature shares for *command*₁ and aggregating them into a full signature. Concurrently, $\text{Cleader}(v_2)$ begins the collection and pre-aggregation of the initial presignature shares for view v_2 , allowing it to start the signing phase as soon as it moves to v_2 . The commitment of a transaction, such as *command*₁, is contingent upon its confirmation in two succeeding views and is finalized in view v_4 , in line with traditional pipelined protocol operations.

This pipelining makes sense, as the signature shares over a view are collected by the cleader into a *topSig*, which is then relayed to the cleader of the next view, essentially delegating responsibility for the next phase. The triggering time slot for the function of *topSig* is in the next view rather than the current view. Therefore, replicas can send signature shares to the current cleader and initial presignature shares to the next cleader for pre-aggregation simultaneously. Although a cleader could act maliciously—by either withholding *topSig* or distorting it—the repercussions are analogous to any traditional consensus protocol malicious leader breach, which could include, for instance, triggering timeouts to initiate the view-change protocol or failing to confirm the current view transaction and thus leaving a “dummy node”.

In addition to pipelining the signature process, ROPSCON extends this to the signing sessions themselves. Each replica i is obligated to not only participate in the current signing activities but also to proactively generate and provide presignature shares. These shares are crucial for ensuring readiness for subsequent signing sessions in either the current or upcoming view. In the current view, this proactive approach is exemplified by the dual responsibility of each replica: it must furnish a signature share, σ_i , for the active session and concurrently supply a presignature share for the next session in a single response (Line 24, 34). This dual submission, addressed to the current view's cleader, effectively establishes a continuous pipeline of signing sessions. As we will prove later, one of the signing sessions per view will eventually terminate after at most $f + 1$ sessions if malicious replicas $\leq f$ and there is a non-faulty cleader, i.e., the cleader receives all the signature shares from votes and can return the final valid signature to construct a proof for next view, thus achieving Robustness. The termination time is bounded after GST.

The benefit of this session-layer pipelining is different from that of standard view-layer pipelining. The latter overlaps the signing phase of current view with the initial pre-signing phase of next view, reducing idle time across consecutive views. In contrast, session-layer pipelining reduces the extra interaction caused by malicious interference inside the view. If a signing session fails due to invalid shares, the cleader can use the presignature shares already provided in previous responses to immediately prepare the next session, instead of launching another separate preprocessing round. Thus, session-layer pipelining further reduces the delay of multi-session signing beyond what standard view-layer pipelining can provide.

Note that it is also feasible to provide presignature shares for multiple future sessions in one response. Since a signing session terminates eventually, carrying more future-session information can reduce additional interactions in the initial stage when many malicious replicas interfere with invalid shares and multiple sessions may be needed (but the lower bound remains one interaction with the current cleader per view, since replicas still need to sign the current view's proposal). However, this optimization increases the communication size, and replicas need to generate and provide multiple signature shares for the same proposal at one time. Moreover, we also prove that the best-case scenario eventually occurs within a finite number of views even with one prepared future session, and in the best case the interaction between replicas and the current cleader is reduced to one per view. Therefore, carrying many future-session presignature shares may gradually become bandwidth and computing power waste as malicious replicas are marked and excluded. The same applies when there are few faults, or when most faults are crash faults.

For the default design that keeps only one future session prepared, in scenarios where malicious replicas is substantial (but still $\leq f$), the efficacy of the pipelining strategy is initially mitigated. This is primarily because it poses a challenge for the incoming cleader to guarantee the exclusion of malicious actors from the set R , especially within the solitary session that transitions through the initial pre-signing phase. In such

Algorithm 3 View-change Protocol

```

1: as Cleader(view), if  $|Malic| > f$  do
2:   newnewview  $\leftarrow$  CreateMsg(VIEWCHANGE,  $\perp$ , view)  $\triangleright$  Active the view-change protocol
3:   broadcast CreateCommun(newnewview,  $\perp$ ,  $\perp$ ,  $\perp$ ,  $\perp$ ,  $\perp$ )
4:   Initialization: Malic  $\triangleright$  Since each replica's Malic is maintained by itself, here only its own Malic is initialized

5: as a replica (replicai) do
6:   upon receiving commun from Cleader(view), isMatch(commun.m, VIEWCHANGE, view)(case=1) or timer timeout (case=2):
7:     stay in view if case = 1, or reset timer and enter view + 1 (view  $\leftarrow$  view + 1) if case = 2
8:     newview  $\leftarrow$  CreateMsg(VIEWCHANGE,  $\perp$ , view + 1)  $\triangleright$  Package the view-change message
9:     ( $\rho'_i$ , staten)  $\leftarrow$  PreRound(PK)  $\triangleright$  Initial presignature share
10:    send CreateCommun(newview, topSig,  $\perp$ ,  $\rho'_i$ ,  $\perp$ ,  $\perp$ ) to Cleader(view + 1)  $\triangleright$  Send to the next cleader

11: as Cleader(view + 1) do
12:   wait for all communications: Commun  $\leftarrow$  {commun | isMatch(commun.m, VIEWCHANGE, view + 1)  $\wedge$  commun.pshare  $\neq$   $\perp$ }
13:   until  $|Commun| = (n - f)$ :
14:   Initialization: topsid, Pshare, Sid, Share, Ra, Pre, R

15:   Sig  $\leftarrow$   $\left( \arg \max_{commun \in Commun \text{ and } \text{Verify}(PK, commun.sig.m, commun.sig.\sigma)} \{commun.sig.m.view\} \right) .sig \triangleright$  highest valid proof
16:   if Sig.m.view > topSig.m.view then topSig  $\leftarrow$  Sig
17:   for commun  $\in$  Commun, whose sender is replicai do
18:     R  $\leftarrow$  R  $\cup$  {i}, Pshare[i]  $\leftarrow$  commun.pshare  $\triangleright$  Store the initial presignature shares
19:      $\rho \leftarrow$  PreAgg(PK, {Pshare[i] | i  $\in$  R})
20:     for i  $\in$  R do Sid[i]  $\leftarrow$  topsid  $\triangleright$  Mark the replica as responsive
21:     leaf  $\leftarrow$  CreateNode(topSig.m.node, command or commands from clients, topSig)  $\triangleright$  Create a leaf node
22:     curProposal  $\leftarrow$  CreateMsg(STATE, leaf, view + 1)  $\triangleright$  Package the node
23:     broadcast CreateCommun(curProposal, topSig,  $\perp$ ,  $\perp$ , R,  $\rho$ )  $\triangleright$  Propose
24:     Ra[topsid]  $\leftarrow$  R, Pre[topsid]  $\leftarrow$   $\rho$   $\triangleright$  Remember the replicas and the presignature
25:   Initialization: R

```

instances, encountering an invalid signature share during the signing phase is more likely, necessitating the establishment of new sessions upon transitioning to the subsequent view. However, as the protocol progresses, the function MarkMalicious systematically marks malicious replicas, increasing the likelihood of the cleader receiving a compliant *R* from a single session in the pre-signing phase, thus enhancing the protocol's efficiency through the pipelining strategy. Over time, this results in a considerable uptick in operational efficiency. It is important to acknowledge, though, the replica that actively triggers a view change will have its *Malic* reset.

Given this pipelining strategy, in the current presentation, it is imperative different replicas serve as cleaders in consecutive views. The initiation of a new presignature session in a new view requires resetting local session-related variables, which would be compromised if a cleader were to oversee two consecutive views. This would jeopardize the integrity of the ongoing signature phase, potentially leading to its failure. However, we note that this is not a fundamental requirement, and is used in the current presentation to simplify the management of session-related variables. ROPSCON can also support the case where the same replica becomes cleader in consecutive views by binding session-related variables to the corresponding view number. Once a view is completed, the session state of this view and earlier views can be garbage-collected. With such view-indexed session management, consecutive views led by the same cleader remain valid, at the cost of slightly more bookkeeping.

View-change Protocol. As shown in Algorithm 3, the activation of the view-change protocol is predicated on two distinct

mechanisms: active invocation via the MarkMalicious function and passive invocation owing to timer timeout (Line 6). The former necessitates that the current cleader take a proactive role by broadcasting view-change messages and **resetting its own *Malic***. The latter is a fail-safe measure, triggered in the absence of timely progress. It's crucial to differentiate the target view implications associated with these triggering conditions. Active invocation of a view-change doesn't inherently imply the subsequent cleader is malicious or down but rather cannot be awakened by the current cleader in the normal state protocol (Fig. 2, Line 47), thereby allowing continuance within the same view after entering the view-change protocol. Conversely, passive triggers imply a more severe condition, suggesting the next cleader is either malicious or down, thus compelling replicas to reset timer and advance to the succeeding view.

In contrast to the normal state protocol, Cleader(*view* + 1) needs to wait for $n - f$ view-change messages from the replicas. It must then select the highest valid proof as the *topSig*, which serves as the foundation for generating leaf proposals and subsequent tree extension. Notably, this procedure does not involve waiting for the wake-up step from the Cleader(*view*)'s message and ultimately transitions back into the normal state protocol.

The proofs for ROPSCON are shown in Appendix B of the supplementary material.

C. Complexity

Synchronous rounds. We consider "synchronous round" as a theoretical concept after GST under partial synchronization.

A “round trip”, which includes a sequence of requests and their corresponding responses, constitutes two synchronous rounds after GST. Therefore, a single synchronous round encapsulates both the cleader broadcasting requests and the non-faulty replicas replying with their responses.

After the initial pre-signing phase, which takes one round, replicas respond to signing phase requests by providing both a signature share for the current session and a presignature share for a possible next session. This pipelining ensures the necessity of only two additional rounds for each session. Since ROPSCON requires at most $f + 1$ sessions to complete a signature, the cleader can present a consensus proof within a minimum of 3 and a maximum of $1 + 2(f + 1) = 2f + 3$ rounds. The pipelining of the signature process also enables the parallel processing of the initial pre-signing phase for view v and the signing phase for view $v + 1$. In fact, only 2 synchronous rounds are required between replicas and the current cleader per view in the best-case scenario.

Communication complexity. We assume that Σ is aggregatable (i.e. supports non-trivial aggregation of presignatures and signatures. FROST3 [21] is aggregatable), the presignatures and signatures maintain the same size as their component shares. During the initial pre-signing phase, each replica sends their votes with initial presignature shares to the cleader once. The cleader then compiles these shares to a presignature and broadcasts a proposal, accompanied by a proof of signature—aggregated from $2f + 1$ valid signature shares representing replicas’ consensus—to all the replicas.

In the signing phase, the cleader exchanges messages with replicas, including presignature shares, presignatures, and signature shares, within the signing session. Given no more than $f + 1$ sessions are requisite for constructing a valid proposal proof, and each session involves $2f + 1$ replicas, the overall communication complexity per proposal in **the worst-case scenario** is $O(n + (f + 1) \cdot (2f + 1))$. This simplifies to $O(f^2)$, and given $n = 3f + 1$, this complexity also translates to $O(n^2)$ with respect to n . In **the best-case scenario**, where only one session is needed, the overall complexity is $O(n)$.

The incremental complexity compared to traditional approaches is justified by ROPSCON’s robustness against malicious votes (signature shares) through concurrent session handling. For instance, if Σ is called directly (without adaptation to the consensus process), while the HotStuff [4] + Σ amalgamation maintains a linear communication complexity of $O(n)$ and requires 4 synchronous rounds, its single-view vote aggregation from $2f + 1$ replicas is vulnerable; a single malicious signature share can disrupt consensus within the view, making its optimal complexity achievable only under the best scenarios. In contrast, ROPSCON not only exhibits consistent best-case complexity, but also guarantees robustness in non-optimal scenarios.

Note that the $O(n^2)$ communication cost corresponds to the non-optimal case where malicious replicas interfere with proof construction and force the cleader to open multiple signing sessions. This does not represent the eventual steady-state cost. As demonstrated in Claim 1 in Appendix B of the supplementary material, views led by non-faulty cleaders automatically

transitions to the best-case scenario once malicious replicas that submit invalid shares are identified and excluded. Moreover, we proved that for any non-faulty cleader whose local *Malic* is not reset, **after a finite number of views led by this cleader, all subsequent views led by it reach the best-case scenario**, even starting with the worst-case. Consequently, views led by non-faulty cleaders eventually operate in the best-case scenario.

VI. EVALUATION

Implementation. We implement ROPSCON in about 5K lines of Rust code, along with the Tokio library for asynchronous IO, and the K256 library for cryptographic operations. We use TCP connections for communication between nodes. The core consensus logic is composed of approximately 1K lines of code, with additional 400 lines for constructing the TS component. The client is implemented as a separate single-threaded process that send transactions to nodes at a specified rate. Nodes read configuration from both JSON files and command line arguments, and output statistics as JSON files for further analysis. We develop a module called CONFIG-GEN to automatically generate configuration files for different experiments, as well as bash scripts to distribute, run, and collect the results across multiple machines for automating the execution. All of the above code is open sourced² on GitHub for reproducibility.

Setup. In our evaluation, we use throughput as our main metric, measured as the number of transactions committed per second. We compared ROPSCON to three HotStuff variants constructed using different signature schemes (also implemented in Rust), respectively. The first variant (referred to as HotStuff-ed25519) uses ed25519 for signing and verification, but instead of using threshold cryptography to aggregate signatures into proofs (also called Quorum Certificates, QC, in HotStuff), it used a collection of individual signatures. The second variant (referred to as HotStuff-bls) uses a modified BLS scheme [9] to aggregate signatures into proofs. The third variant (referred to as HotStuff-frost3) calls FROST3 directly (without adaptation to the consensus process) to construct proofs. Among them, ed25519 is a fast and efficient signature scheme. BLS is a well-known multisignature scheme requiring only one round of interaction, which is widely used in various implementations. FROST3 is used as the underlying scheme of ROPSCON, and we also apply it to HotStuff for comparison.

Experiments were conducted on a cluster of five Alibaba Cloud ECS g6.3xlarge instances, each with 12 virtual CPUs (vCPUs) on a 2.5GHz, Intel Xeon Platinum 8269CY (Cascade Lake), 48GB memory, ran Ubuntu 22.04 LTS and deployed rustc 1.67.0-nightly. Replicas were distributed across those instances equally. We particularly aim to demonstrate: (1) ROPSCON achieves higher throughput compared to both HotStuff-bls and HotStuff-ed25519, particularly evident in high-delay and low-bandwidth environments, capitalizing on the advantages of the Schnorr paradigm scheme; (2) through redundancy avoidance and pipelining techniques, ROPSCON significantly reduces the interaction rounds between replicas and the current leader, providing considerable throughput improvements

²<https://github.com/BerserkRugal/ROPSCON>

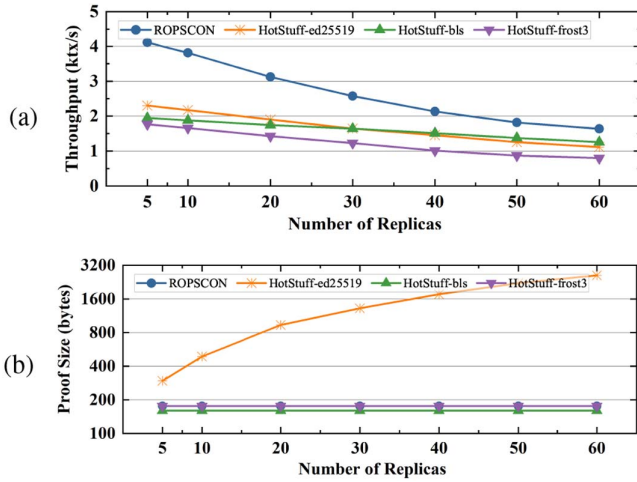


Fig. 5. Throughput and proof size of ROPSCON and HotStuff variants for varying number of replicas from 5 to 60. No faults, a transaction size of 128 bytes, a batch size of 1000, a RTT of 200 ms and a bandwidth of 25Mbps.

compared to direct integration approaches (HotStuff-frost3); (3) ROPSCON maintains robust performance when malicious replicas attempt to disrupt consensus proof construction by sending invalid signature shares.

In the subsequent part of this section, each measurement in the graphs is the average of 10 independent runs, and unless otherwise specified, the batch size is set to 1000 transactions per node, the transaction size is 128 bytes and the timeout is 20s for each replica. When referring to latency, we mean the time a client generate a transaction to the time the transaction is committed by a replica.

Performance in the global scenario. We model a globally distributed blockchain characterized by a round-trip time (RTT) of 200ms and a bandwidth of 25Mbps³. Similar settings can also be found in [17], which reflects real-world wide-area deployments, such as geo-distributed permissioned blockchains, cross-region data-center replication, and cloud-edge systems. In these scenarios, replicas may be located in different regions, where communication latency is dominated by long-distance network transmission and bandwidth can be constrained by private links, inter-cloud bandwidth limits, or operational costs. Fig. 5(a) and 5(b) illustrate the throughput and the proof size of ROPSCON and HotStuff variants for varying numbers of replicas, respectively.

HotStuff-ed25519. With 5 replicas, HotStuff-ed25519 achieves a maximum throughput of around 2300 tx/s, which drops to approximately 1100 tx/s when the number of replicas is increased to 60. Although ed25519 has a high signature and verification speed, the lack of signature aggregation results in a significant increase in proof size with the number of replicas. At 60 replicas, the proof size approaches the size of 20 transactions (~2600 bytes), leading to its mediocre performance when bandwidth is limited and delay is high.

³The RTT and bandwidth settings in our experiments were emulated using Linux traffic control (tc) with netem.

HotStuff-bls. HotStuff-bls has lower throughput with 30 replicas or less than HotStuff-ed25519 due to additional overhead of signature aggregation and verification. However, as replicas increases, it gradually shows an advantage because it reduces the proof size to a constant 160 bytes. The throughput stays around 1250 tx/s at 60 replicas, during which the cost of signature aggregation is lower than the cost of additional proof size and proof verification time of HotStuff-ed25519.

HotStuff-frost3. Since frost3 is called directly (without adaptation to the consensus process), HotStuff-frost3 needs an extra round of presignature aggregation between the leader and replicas, leading to a poor performance in high-delay and low-bandwidth environments. The throughput decreases from 1700 tx/s with 5 replicas to around 800 tx/s with 60 replicas.

ROPSCON. ROPSCON also maintains a constant proof size of 176 bytes. It performs better in high-delay, low-bandwidth environments, achieving a throughput of approximately 4100 tx/s with 5 replicas and 1700 tx/s with 60 replicas. This result is an approximate $1.5 \times - 1.7 \times$ improvement compared to HotStuff-ed25519 and an $1.4 \times - 2.2 \times$ improvement compared to HotStuff-bls, capitalizing on the advantages of the pairing-free and computationally efficient Schnorr TS scheme in constructing linear BFT.

Performance under bandwidth and delay constraints lifted.

Fig. 6 illustrates the latency and throughput of ROPSCON and HotStuff variants for varying numbers of replicas under bandwidth and delay constraints lifted (a RTT of 10ms and a bandwidth of 1000Mbps). ROPSCON peaks at 70,000 tx/s with 10 replicas, exhibiting a slightly lower throughput than HotStuff-ed25519. However, as replicas increases to 20, its throughput (up to 59,000 tx/s) becomes comparable, and even surpasses HotStuff-ed25519 at 50 replicas (around 31,000 tx/s). However, its latency before saturation remained a little higher than that of HotStuff-ed25519 (despite ROPSCON pipelining the presignature process, the additional step still contributes to the overall delay).

When compared to variants with threshold signature integration, ROPSCON demonstrates a clear advantage. The maximum throughput observed for HotStuff-bls is 31,000 tx/s (10 replicas) and 13,000 tx/s (50 replicas), less than half that of ROPSCON. Additionally, its latency is higher, reaching 120ms at 50 replicas. The performance of HotStuff-frost3 is even worse. These comparisons show that ROPSCON performs well in regular environments. Its redundancy avoidance, along with its pipelined approach, significantly contributes to the improvement of throughput.

Performance under malicious signature shares interference.

Fig. 7 depicts the performance of HotStuff-bls, HotStuff-frost3, and ROPSCON when a total of 10 replicas suffers 1 to 3 malicious-faults (i.e. sending invalid signature shares (votes) for any proposal to interfere with the consensus, while still following the normal consensus process, e.g. proposing correctly. Such behaviour is for robustness evaluation, and is different from crash-faults). HotStuff-ed25519 is not compared as it does not use TS and is not affected by invalid votes. The achieved robustness allows ROPSCON to maintain a peak throughput of around 62,500 tx/s with a latency of approximately 50 ms

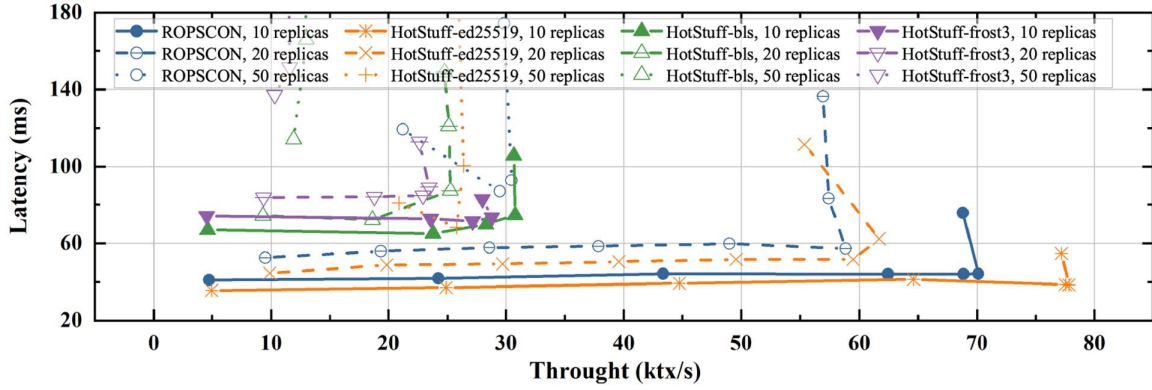


Fig. 6. Comparative throughput-latency performance of ROPSCON and HotStuff variants with 10, 20 and 50 replicas. No faults, a transaction size of 128 bytes, a batch size of 1000, a RTT of 10 ms and a bandwidth of 1000Mbps.

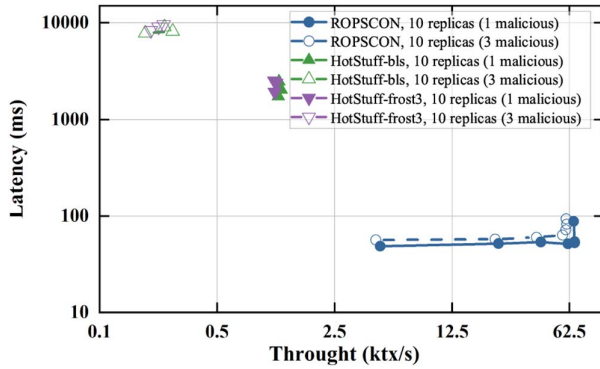


Fig. 7. Comparative throughput-latency performance of ROPSCON and HotStuff variants with 10 replicas. 1 or 3 malicious replicas sending invalid shares to interfere. A transaction size of 128 bytes, a batch size of 1000, a RTT of 10 ms and a bandwidth of 1000Mbps.

(1 faults) and 60 ms (3 faults), which is only a slight decrease compared to no malicious share interference. In contrast, the performance of HotStuff-bls and HotStuff-frost3 significantly decreases. The throughput drops below 1,200 tx/s with only 1 fault (a decrease of over 10x compared to no faults), and with 3 faults, the throughput is only around 200 tx/s, while latency is even close to 10s. Consensus is reached only when $2f + 1$ votes collected for a proposal are all valid, due to the lack of robustness.

VII. CONCLUSION

This paper presented ROPSCON, the first linear BFT protocol effectively utilizing Schnorr TS. ROPSCON addresses the two main obstacles of integrating these computationally efficient, pairing-free signatures into BFT—invalid-share disruption and multi-round interaction—through three main contributions: a novel framework enabling robust proof construction, tight integration of the signature process with the consensus flow, and dual-level pipelining across views and sessions. Experiments show that ROPSCON achieves up to $2.2\times$ higher

throughput compared to HotStuff variants using BLS or non-aggregated signatures, particularly under high-latency and low-bandwidth conditions, while maintaining robust performance against invalid-share interference by malicious replicas.

REFERENCES

- [1] X. Wang, S. Duan, J. Clavin, and H. Zhang, “BFT in blockchains: From protocols to use cases,” *ACM Comput. Surv.*, vol. 54, no. 10s, pp. 1–37, 2022.
- [2] M. Belotti, N. Božić, G. Pujolle, and S. Secci, “A vademecum on blockchain technologies: When, which, and how,” *IEEE Commun. Surveys Tuts.*, vol. 21, no. 4, pp. 3796–3838, 4th Quart. 2019.
- [3] C. Wu, M. J. Amiri, H. Qin, B. Mehta, R. Marcus, and B. T. Loo, “Towards full stack adaptivity in permissioned blockchains,” *Proc. VLDB Endowment*, vol. 17, no. 5, pp. 1073–1080, 2024.
- [4] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham, “HotStuff: BFT consensus with linearity and responsiveness,” in *Proc. ACM Symp. Princ. Distrib. Comput.*, 2019, pp. 347–356.
- [5] G. G. Gueta et al., “SBFT: A scalable and decentralized trust infrastructure,” in *Proc. 49th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 568–580.
- [6] E. Buchman, “Tendermint: Byzantine fault tolerance in the age of blockchains,” *Ph.D. dissertation, University of Guelph*, 2016. [Online]. Available: <http://hdl.handle.net/10214/9769>
- [7] V. Buterin and V. Griffith, “Casper the friendly finality gadget,” 2017. [Online]. Available: <https://arxiv.org/abs/1710.09437>
- [8] D. Boneh, B. Lynn, and H. Shacham, “Short signatures from the Weil pairing,” in *Proc. Int. Conf. Theory Appl. Cryptology Inf. Secur.*, New York, NY, USA: Springer-Verlag, 2001, pp. 514–532.
- [9] D. Galindo, J. Liu, M. Ordean, and J.-M. Wong, “Fully distributed verifiable random functions and their application to decentralised random beacons,” in *Proc. IEEE Eur. Symp. Secur. Privacy*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 88–102.
- [10] R. Bacho and J. Loss, “On the adaptive security of the threshold bls signature scheme,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 193–207.
- [11] D. R. Stinson and R. Strohli, “Provably secure distributed Schnorr signatures and a (t, n) threshold scheme for implicit certificates,” in *Proc. Australas. Conf. Inf. Secur. Privacy*, New York, NY, USA: Springer-Verlag, 2001, pp. 417–434.
- [12] R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin, “Secure distributed key generation for discrete-log based cryptosystems,” *J. Cryptology*, vol. 20, no. 1, pp. 51–83, Jan. 2007.
- [13] C. Komlo and I. Goldberg, “Frost: flexible round-optimized Schnorr threshold signatures,” in *Proc. 27th Int. Conf. Selected Areas Cryptography*, New York, NY, USA: Springer-Verlag, 2021, pp. 34–65.
- [14] E. Crites, C. Komlo, and M. Maller, “How to prove Schnorr assuming Schnorr: Security of multi-and threshold signatures,” *Cryptology ePrint Archive*, 2021. [Online]. Available: <https://eprint.iacr.org/2021/1375.pdf>
- [15] F. Benhamouda, S. Halevi, H. Krawczyk, Y. Ma, and T. Rabin, “Sprint: High-throughput robust distributed Schnorr signatures,” in *Proc. Annu.*

- Int. Conf. Theory Appl. Cryptographic Techn.*, New York, NY, USA: Springer-Verlag, 2024, pp. 62–91.
- [16] R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin, “Secure distributed key generation for discrete-log based cryptosystems,” in *Proc. Adv. Cryptology—EUROCRYPT Int. Conf. Theory Appl. Cryptographic Techn.*, New York, NY, USA: Springer-Verlag, 1999, pp. 295–310.
 - [17] R. Neiheiser, M. Matos, and L. Rodrigues, “Kauri: Scalable BFT consensus with pipelined tree-based dissemination and aggregation,” in *Proc. ACM SIGOPS 28th Symp. Operating Syst. Princ.*, 2021, pp. 35–48.
 - [18] R. Gelashvili, L. Kokoris-Kogias, A. Sonnino, A. Spiegelman, and Z. Xiang, “Jolteon and ditto: Network-adaptive efficient consensus with asynchronous fallback,” in *Proc. Int. Conf. Financial Cryptography Data Secur.*, New York, NY, USA: Springer-Verlag, 2022, pp. 296–315.
 - [19] I. Doidge, R. Ramesh, N. Shrestha, and J. Tobkin, “Moonshot: Optimizing block period and commit latency in chain-based rotating leader BFT,” in *Proc. 54th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, Piscataway, NJ, USA: IEEE Press, 2024, pp. 470–482.
 - [20] Q. Feng, K. Yang, M. Ma, and D. He, “Efficient multi-party EdDSA signature with identifiable aborts and its applications to blockchain,” *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 1937–1950, 2023.
 - [21] T. Ruffing, V. Ronge, E. Jin, J. Schneider-Bensch, and D. Schröder, “Roast: Robust asynchronous Schnorr threshold signatures,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 2551–2564.
 - [22] L. Lamport, R. Shostak, and M. Pease, “The byzantine generals problem,” in *Proc. Concurrency, Works Leslie Lamport*, 2019, pp. 203–226.
 - [23] E. Blum, J. Katz, J. Loss, K. Nayak, and S. Ochseneither, “Abraxas: Throughput-efficient hybrid asynchronous consensus,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2023, pp. 519–533.
 - [24] M. J. Fischer, N. A. Lynch, and M. S. Paterson, “Impossibility of distributed consensus with one faulty process,” *J. ACM*, vol. 32, no. 2, pp. 374–382, 1985.
 - [25] M. Ben-Or, “Another advantage of free choice (extended abstract) completely asynchronous agreement protocols,” in *Proc. 2nd Annu. ACM Symp. Princ. Distrib. Comput.*, 1983, pp. 27–30.
 - [26] C. Dwork, N. Lynch, and L. Stockmeyer, “Consensus in the presence of partial synchrony,” *J. ACM*, vol. 35, no. 2, pp. 288–323, 1988.
 - [27] M. Castro and B. Liskov et al., “Practical byzantine fault tolerance,” in *Proc. 3rd Symp. Oper. Syst. Des. Implementation*, 1999, vol. 99, pp. 173–186.
 - [28] I. Abraham, D. Malkhi, K. Nayak, L. Ren, and M. Yin, “Sync HotStuff: Simple and practical synchronous state machine replication,” in *Proc. IEEE Symp. Secur. Privacy*, Piscataway, NJ, USA: IEEE Press, 2020, pp. 106–118.
 - [29] X. Liu et al., “Dolphin: Efficient non-blocking consensus via concurrent block generation,” *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 11824–11838, Dec. 2024.
 - [30] S. Ling, Z. Liu, Q. Li, X. Du, J. Chen, and K. Xu, “Stable byzantine fault tolerance in wide area networks with unreliable links,” *IEEE/ACM Trans. Netw.*, vol. 32, no. 6, pp. 5310–5325, Dec. 2024.
 - [31] I. Keidar, E. Kokoris-Kogias, O. Naor, and A. Spiegelman, “All you need is DAG,” in *Proc. ACM Symp. Princ. Distrib. Comput.*, 2021, pp. 165–175.
 - [32] A. Spiegelman, N. Giridharan, A. Sonnino, and L. Kokoris-Kogias, “Bullshark: DAG BFT protocols made practical,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 2705–2718.
 - [33] B. Arun, Z. Li, F. Suri-Payer, S. Das, and A. Spiegelman, “Shoal++: High throughput {DAG} {BFT} can be fast and robust!,” in *Proc. 22nd USENIX Symp. Netw. Syst. Des. Implementation*, 2025, pp. 813–826.
 - [34] J. Nick, A. Poelstra, and G. Sanders, “Liquid: A bitcoin sidechain,” *Liquid White Paper*, 2020. [Online]. Available: <https://blockstream.com/assets/downloads/pdf/liquid-whitepaper.pdf>
 - [35] S. Goldfeder et al., “Securing bitcoin wallets via a new DSA/ECDSA threshold signature scheme,” 2015. [Online]. Available: http://stevengoldfeder.com/papers/threshold_sigs.pdf
 - [36] X. Liu et al., “The deferred byzantine generals problem,” *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 7777–7792, 2025.
 - [37] R. Gennaro and S. Goldfeder, “Fast multiparty threshold ECDSA with fast trustless setup,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 1179–1194.
 - [38] Y. Lindell and A. Nof, “Fast secure multiparty ECDSA with practical distributed key generation and applications to cryptocurrency custody,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 1837–1854.
 - [39] J. Doerner et al., “Threshold ECDSA in three rounds,” in *Proc. IEEE Symp. Secur. Privacy*, Piscataway, NJ, USA: IEEE Press, 2024, pp. 175–175.
 - [40] J. Doerner, Y. Kondi, E. Lee, and A. Shelat, “Threshold ECDSA from ECDSA assumptions: The multiparty case,” in *Proc. IEEE Symp. Secur. Privacy*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 1051–1066.
 - [41] M. Bellare, S. Tessaro, and C. Zhu, “Stronger security for non-interactive threshold signatures: BLS and FROST,” *Cryptology ePrint Archive*, 2022. [Online]. Available: <https://eprint.iacr.org/2022/833.pdf>
 - [42] S. Joshi, D. Pandey, and K. Srinathan, “ATSSIA: Asynchronous truly-threshold Schnorr signing for inconsistent availability,” in *Proc. Int. Conf. Inf. Secur. Cryptology*, New York, NY, USA: Springer-Verlag, 2021, pp. 71–91.
 - [43] Y. Lindell, “Simple three-round multiparty Schnorr signing with full simulatability,” *Cryptology ePrint Archive*, 2022. [Online]. Available: <https://eprint.iacr.org/2022/374.pdf>
 - [44] X. Liu et al., “ABSE: Adaptive baseline score-based election for leader-based BFT systems,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 36, no. 8, pp. 1634–1650, Aug. 2025.
 - [45] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” in *Proc. Concurrency, Works Leslie Lamport*, 2019, pp. 179–196.
- Xuyang Liu** received the B.E. degree in computer science and technology from Beijing Institute of Technology in 2022. He is currently working toward the Ph.D. degree both with the School of Cyberspace Science and Technology, Beijing Institute of Technology, and the School of Computer Science, University of Auckland. His research interests include applied cryptography, blockchain technology, and distributed consensus.
- Zijian Zhang** (Senior Member, IEEE) is a Professor with the School of Cyberspace Science and Technology, Beijing Institute of Technology. He is also a Research Fellow with the School of Computer Science, University of Auckland. He was a Visiting Scholar with the Computer Science and Engineering Department, State University of New York at Buffalo in 2015. His research interests include design of authentication and key agreement protocol and analysis of entity behavior and preference.
- Haibin Zhang** received the Ph.D. degree in computer science from the University of California, Davis, in 2014. He is currently the Director with the Department of Information Technology, Yangtze Delta Region Institute of Tsinghua University, Zhejiang and a Professor with Beijing Institute of Technology. His research interests include intersection of distributed systems, system security, and applied cryptography.
- Jiamou Liu** received the Ph.D. degree in computer science from the University of Auckland in 2010. He is currently an Associate Professor with the School of Computer Science, University of Auckland. He was a Senior Lecturer with Auckland University of Technology, Auckland, from 2011 to 2015 and a Researcher with the Department of Computer Science, Leipzig University, Leipzig, Germany, from 2009 to 2010. His research interests include social network analysis, multiagent systems, and algorithms.
- Meng Li** (Senior Member, IEEE) is a Professor with the School of Computer Science and Information Engineering, Hefei University of Technology. He received the Ph.D. degree in computer science and technology from Beijing Institute of Technology in 2019. He was a Postdoctoral Researcher with the Department of Mathematics and HIT Center, University of Padua, where he is with the Security and Privacy Through Zeal (SPRITZ) Research Group led by Prof. Mauro Conti (IEEE Fellow). He was sponsored by ERCIM ‘Alain Bensoussan’ Fellowship Programme (2020.10–2021.3) to conduct Postdoctoral Research with Prof. Fabio Martinelli at CNR, Italy. Supported by China Scholarship Council (CSC), he was a joint Ph.D. student (2017.9–2018.8) with Prof. Xiaodong Lin (IEEE Fellow) in the Broadband Communications Research (BBRC) Lab, at the University of Waterloo and Wilfrid Laurier University, and a visiting scholar (2025.3–2025.6) with Prof. Mauro Conti at the HIT Center, University of Padua. His research interests include secure and trustworthy data sharing, privacy preservation in IoV, applied cryptography, blockchain, TEE. In these areas, he has published 173 papers in topmost journals and conferences, including IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING, *ACM Transactions on Database Systems*, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, IEEE TRANSACTIONS ON COMPUTERS, S&P, USENIX Security, MobiCom, and INFOCOM. He is a Senior Member of CCF, CACR, CIE, and CIC; a SAE

for TIFS; an AE for TDSC; and has served as a TPC Member for conferences such as ICDCS, Inscrypt, ACISP, ICICS, and TrustCom. He receives 2024 IEEE HITC Award for Excellence (Early Career Researcher), 2025 IEEE TCSVC Rising Star Award and 2026 IEEE TCHS Young Researcher Award. He was selected into the IEEE Computer Society Computing's Top 30 Early Career Professionals for 2025.

Zhen Li (Member, IEEE) is currently working toward the Ph.D. degree with the School of Cyberspace Science and Technology, Beijing Institute of Technology. His research interests include privacy computing, AI security, and blockchain.

Mengmeng Zhou received the bachelor degree in computer science and technology from the School of Computer Science, Beijing University of Technology. He serves as an Architect of Beijing Academy of Blockchain and Edge Computing, BAEC, Beijing, the Chief Architect of Privacy Computing Product with BAEC. He has nearly 20 years experience in software development and complex system architecture design. He worked for many large manufacturers in the IT industry, such as HPIBM, Pirangut, India, and Huawei, Shenzhen, China. He is an expert of communications technology, electronic commerce, cloud, blockchain, and privacy computing area.

Wei Shen received the master degree in communications and information systems from the School of Electronic Engineering, Beihang University, Beijing. He serves as a Privacy Computing Researcher of Beijing Academy of Blockchain and Edge Computing, BAEC, Beijing. He focuses on Privacy Computing Product with BAEC. He has more than 20 years experience in software development and complex system architecture design. He worked for many large manufacturers in the IT industry, such as Bell Labs/Lucent and Spirent Commubniations. Expert of communications technology, electronic commerce, cloud, blockchain, and privacy computing area.

Peng Jiang received the Ph.D. degree from Beijing University of Posts and Telecommunications in 2017. She is an Associate Professor with the School of Cyberspace Science and Technology, Beijing Institute of Technology. She has edited three book/book chapters and published more than 50 refereed papers in international journals and conferences, including one ESI Highly Cited Papers (2020), one ESI Hot Paper (2021), and one Editor's Choice Paper. Her research interests include cryptography, information security, and blockchain technology. She has served as an Associate Editor for the Computer Standards & Interfaces, a guest editor for many journals, and a program committee member of many conferences. She also serves as the Co-Chair of SIG for the IEEE TEMS's TC on Blockchain and Distributed Ledger Technologies.

Cong Guo is a Doctor of Philosophy with the National Meteorological Information Centre. Her research interests include group key exchange, cloud storage security and Internet of Vehicles.

Liehuang Zhu (Senior Member, IEEE) is a Professor with the School of Cyberspace Science and Technology, Beijing Institute of Technology. He is selected into the Program for New Century Excellent Talents in University from Ministry of Education, P. R. China. His research interests include cryptographic algorithms and secure protocols, Internet of Things security, cloud computing security, big data privacy, mobile and Internet security, and trusted computing.